



Trabajo Fin de Máster

Máster Universitario en Lógica,
Computación e Inteligencia Artificial

Departamento de Ciencias de la Computación e Inteligencia Artificial

Clasificación de imágenes multi-etiqueta en entornos federados

Autor/a:
D. Manuel Mateos Gutiérrez

Tutor:
Dr. Jose María Moyano Murillo

Sevilla, XX de XXXX de 20XX

Índice general

Índice de figuras **V**

Índice de tablas **VII**

1. Introducción **1**

2. Estado del arte **5**

2.1. Aprendizaje Federado 5

2.1.1. Aprendizaje federado heterogéneo 7

2.2. Clasificación multi-etiqueta 8

2.2.1. Descripción formal del MLC 9

2.2.2. Eligiendo modelo para clasificación multi-etiqueta 10

3. Federación con capas personalizadas y salidas heterogéneas (FedPerH) **11**

3.1. Datos de entrenamiento 11

3.2. Arquitectura de la solución 12

3.2.1. FedPer 12

3.2.2. Modelo 13

3.2.3. Modificaciones 13

3.3. Métricas de evaluación 14

3.3.1. Label ranking average precision 14
3.3.2. Matriz de confusión 15
3.3.3. F1 Score 18

4. Estudio experimental **19**

4.1. Configuración de los experimentos 19
4.2. Análisis de resultados 20

5. Conclusiones y trabajo futuro **21**

5.1. Conclusiones 21
5.2. Trabajo futuro 22

Bibliografía **23**

Índice de figuras

2.1. Entrenamiento federado cliente-servidor 6

2.2. Clasificación binaria y multi-clase 8

2.3. Clasificación multi-etiqueta 9

2.4. Ejemplo de conjunto de datos multi-etiqueta obtenido de [1] 9

Índice de tablas

1. Introducción

Vivimos una época de auge en el campo de la Inteligencia Artificial provocado por la llegada de nuevas tecnologías en modelos de lenguaje y sistemas generativos que han creado un aumento en la demanda y la inversión, lo que ha ocasionado que se desarrollen aún más estas tecnologías, se haya normalizado su uso y se estén integrando en muchos y variados ámbitos tanto cotidianos (agentes de voz y de texto, asistentes, buscadores...) como en los negocios (asistentes a la programación, automatizaciones, sistemas integrados en medicina e investigación, logística, hostelería...).

Este florecimiento ha traído consigo también nuevos desafíos tanto en arquitectura como en regulaciones. Que cada vez haya más dispositivos recolectando y produciendo datos necesarios para entrenar modelos hace que estos sistemas tengan que adaptarse para ser más flexibles y poder dar cobertura a diferentes tipos de topologías, por ejemplo, varios dispositivos móviles pueden colaborar para entrenar una red teniendo diferentes especificaciones. Asimismo la compartición de datos personales e información sensible trae consigo la necesidad de crear sistemas más seguros, por ejemplo una persona que esté entrenando un modelo con su geolocalización pero sin tener que compartir su posición con una empresa externa. Este tipo de problemas son el germen para el desarrollo de sistemas de aprendizaje federados [2].

La idea detrás de esta arquitectura es que varios clientes entrenen modelos locales y compartan los parámetros (por ejemplo, los pesos de la red) con un servidor de manera que todos cooperan en el entrenamiento conjunto de la red mientras que los datos nunca abandonan los clientes. Podemos imaginar un entorno en el que los clientes tienen datos sensibles (datos en un dispositivo personal o de pacientes en un centro sanitario) que no se desea compartir con una empresa externa o que no pueden salir de una determinada zona como por ejemplo datos de empresas de la Unión Europea que no se pueden compartir con servidores fuera de ella. A parte de la privacidad, otro beneficio es que los recursos requeridos para los entrenamientos locales son menores comparados con una estructura centralizada, de manera que el resultado de la cooperación de muchos clientes es más eficiente

que si trabajasen de manera aislada con esos mismos recursos. También es importante mencionar que de esta forma se permite de manera más fácil la integración de nuevos clientes (dispositivos, empresas, entidades públicas...) con sistemas ya establecidos.

Como caso de uso podemos plantear un entorno en el que varios centros médicos de diferentes disciplinas quieren hacer uso de un sistema de procesado de imágenes médicas sin que estas abandonen los centros. El coste de que cada centro entrene su propio modelo es alto y requeriría un gran número de datos de entrenamiento. Para que todos los centros pudieran colaborar en entrenar un mismo modelo tendrían que agregar las imágenes médicas en un mismo servidor y esto crearía problemas de privacidad y políticas de tratamiento de datos. Un sistema federado puede solucionar estos problemas ya que los datos no abandonan los clientes y al participar todos ellos en el entrenamiento común el resultado que obtienen es mayor que el aporte que hacen al entrenamiento por separado, por lo que los recursos son usados de manera más eficiente.

Sin embargo aún tendríamos un problema y es el de la especialización de cada cliente, en el supuesto actual todos ellos deberían realizar la misma tarea pero podría haber casos en los que las tareas específicas sean diferentes para cada uno, aun compartiendo características comunes en el aprendizaje. Por ejemplo, varios clientes quieren realizar clasificación de imágenes, pero cada uno tiene fotos de distinta tipología o quieren detectar diferentes conjuntos de objetos. Mientras que la tarea es común en todos, las clases o etiquetas a predecir son específicas de cada cliente.

Además de todos sus beneficios aún existen varios desafíos en el campo del aprendizaje federado. La bondad de los modelos federados está probada para datos homogéneos, pero cuando los datos son consistentes en etiquetas, características o cantidad es más fácil que los clientes al entrenarse lleguen a un máximo global. Sin embargo si hay desequilibrios en estas categorías es más fácil que lleguen a un máximo local, lo que dificulta la agregación de los pesos de las distintas redes de los clientes en el servidor.

En nuestro caso particular analizaremos la viabilidad de implementar un modelo de clasificación multi-etiqueta (donde cada ejemplo puede categorizarse en más de una clase o etiqueta al mismo tiempo) con datos heterogéneos dentro de un sistema federado. La idea es que cada cliente sea usado para una tarea diferente entrenando solo un subconjunto de etiquetas y se pueda beneficiar del aporte del resto de clientes.

El resto de la memoria se organiza del siguiente modo:

2. Estado del arte. En este capítulo se analizarán qué sistemas existen que implementen clasificación multi-etiqueta en entornos federados o qué estudios se

acercan. También se definirá la base da la que partiremos para nuestra implementación.

3. Propuesta. Se describen los sistemas desarrollados en este trabajo.

4. Estudio Experimental. Se presentan los resultados de los experimentos realizados.

5. Conclusiones y Trabajo Futuro. Se plantean las conclusiones y futuras ampliaciones del sistema y líneas de trabajo futuro en este campo.

2. Estado del arte

En esta sección explicaremos los elementos necesarios para la resolución del problema propuesto. Haremos una explicación en profundidad de los sistemas de aprendizaje federados y de la clasificación multi-etiqueta.

2.1. Aprendizaje Federado

En aprendizaje automático clásico tenemos un modelo que entrenamos con una serie de datos para que realice una tarea que puede ser reconocer objetos en una imagen, controlar la reacción de un sistema ante una situación o jugar a un juego. En la práctica estos datos no se generan en la máquina dónde entrenamos al modelo sino que pueden venir de un dispositivo móvil, unos sensores o un micrófono. Estos datos también pueden venir no de una única fuente sino de varios lugares que recopilen información para realizar una tarea en conjunto. La aproximación más común ante esta estructura de datos es unir todas las fuentes en un lugar centralizado y tratarlas para crear un único conjunto de datos y usarlo para realizar el entrenamiento de nuestro modelo.

Este enfoque no puede usarse en todos los casos, por ejemplo cuando los datos no pueden compartirse con el servidor central debido a regulaciones, preferencias de privacidad o simplemente que el volumen de datos es tan grande que el servidor no es capaz de procesarlos todos entre muchos otros ejemplos.

Frente a estos problemas nace el paradigma de aprendizaje federado [2] el cuál invierte esta estructura de manera que lleva el entrenamiento a los datos. Esto nos permite aplicar el campo de aplicación de los sistemas de aprendizaje automático a lugares donde antes era difícil o imposible debido a la naturaleza de los datos como detección de fraude financiero haciendo que varias entidades bancarias cooperen en el aprendizaje o mejorar modelos médicos agregando el entrenamiento de diferentes sistemas sanitarios sin que las normas de privacidad se vean comprometidas.

Hay varias aproximaciones como la arquitectura *peer to peer* donde todos los nodos comparten entre ellos el resultado del entrenamiento de los demás. Nosotros nos centraremos en explicar el aprendizaje federado en arquitectura cliente-servidor. Con este método primero se inicializa el modelo global con pesos aleatorios y se envía a todos los clientes, de esta manera nos aseguramos de que todos los clientes partan de los mismos parámetros. Luego cada cliente entrena su modelo local, que es una copia del modelo global con sus datos locales. Este entrenamiento no se realiza hasta la convergencia del modelo, sino solo realiza unos pocos pasos de entrenamiento. Una vez actualizado el modelo local de cada cliente estos envían la actualización al servidor ya sean los pesos o los gradientes. El servidor recibe una versión diferente del modelo inicial por cada cliente que ha participado en la ronda de entrenamiento. En este punto recordamos que el objetivo es que los clientes colaboren con sus entrenamientos parciales a actualizar un modelo global, para ello el servidor realiza una operación de unificación de todas las versiones del modelo que le han mandado mediante un proceso que llamamos agregación. El método más básico se conoce como *Federated Averaging* [3] abreviado como *FedAvg*. Este proceso realiza una media ponderada sobre los pesos de los diferentes modelos en base al número de ejemplos que han usado en el entrenamiento para asegurarse de que cada ejemplo que ha participado tiene la misma influencia. Una vez finalizada la agregación el servidor envía el nuevo modelo a cada uno de los clientes. Este proceso se vuelve a ejecutar en bucle hasta que los modelos locales alcanzan la convergencia o un número de rondas prefijado. Este proceso está ilustrado en la figura 2.1.

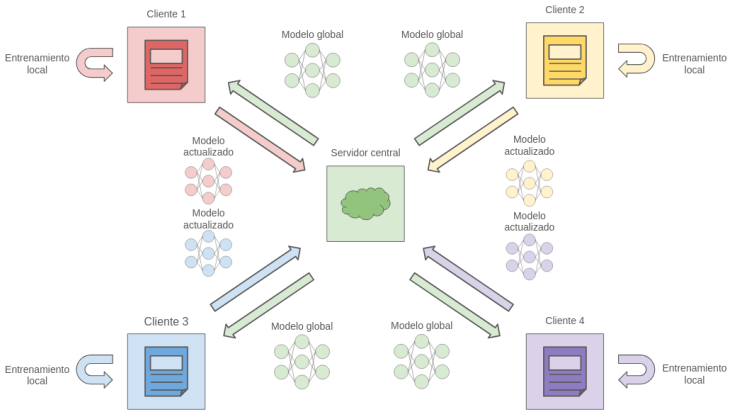


Figura 2.1: Entrenamiento federado cliente-servidor

2.1.1. Aprendizaje federado heterogéneo

En [4] se realiza una clasificación de los problemas presentes en las diferentes arquitecturas de modelos federados según si la heterogeneidad es estadística, en el modelo, en las comunicaciones o en los dispositivos. Si hablamos de la heterogeneidad estadística estamos hablando de los datos y el principal problema es que no sean independientes e identicamente distribuídos (no-IID) lo que hace que el aprendizaje se reduzca considerablemente [5] [6]. Ahondando más en esta categorización pueden tener desbalance de etiquetas, desbalance de características, desbalance de calidad y desbalance de cantidad. Artículos como [7] hablan de este problema en datos médicos y atacan el problema centrándose en la normalización de los datos.

El estudio también evalúa otros tipos de heterogeneidad en la comunicación, los dispositivos y los modelos y propone diferentes soluciones para mitigar estos problemas. El caso que nos compete tiene desbalance en etiquetas ya que como hemos comentado antes cada cliente va a entrenar una tarea distinta, por lo que los datos de cada uno solo tendrán un subconjunto de las etiquetas y también desbalance en los modelos ya que cada cliente va a tener un número de etiquetas diferente por lo que la salida de la red local no será consistente entre todo el sistema. En cuanto al desbalance en el modelo se proponen varias soluciones, una de ellas es la compartición de arquitectura, que consiste en compartir parte del modelo y hacer un entrenamiento personalizado, este método es capaz de solucionar tanto la heterogeneidad estadística como la de modelo y de dispositivo al mismo tiempo [8].

Los métodos a nivel de datos pueden resolver el problema asociado a las grandes diferencias en la distribución de los datos y acelerar la convergencia reduciendo heterogeneidad estadística entre ellos. Por el contrario, los métodos a nivel de modelo buscan aprender un modelo local para cada cliente que se adapte a su distribución de datos privada como *Backbone Sharing* que únicamente comparte una parte del modelo manteniendo capas personalizadas en el cliente, esta solución es la que queremos estudiar en este trabajo.

2.2. Clasificación multi-etiqueta

Una de las tareas más extendidas en el ámbito del aprendizaje automático es la de la clasificación ya sea de textos, imágenes o cualquier otro tipo de conjunto de información. Consiste en crear un modelo en base a las características del conjunto de datos que vayamos a procesar para que realice una predicción sobre las clases asignadas a esos datos. Los ejemplos más comunes son la clasificación binaria y la clasificación multi-clase 2.2. En la clasificación binaria asignamos un valor binario a los datos de entrada, por ejemplo un sistema que detecte si un correo es spam o no. La clasificación multi-clase define una serie de elementos de los cuales se debe elegir el que sea más probable que sea relacionado con los datos que estamos evaluando, por ejemplo detectar qué tipo de animal aparece en una foto.



Figura 2.2: Clasificación binaria y multi-clase

En el mundo real existen muchos escenarios donde se requiere una clasificación múltiple en la misma tarea, por ejemplo detectar diferentes patologías en una imagen médica, añadir distintas categorías o temas a un texto, detectar diferentes objetos en un vídeo en tiempo real etcétera. En situaciones donde un mismo conjunto de datos pueden clasificarse con más de una clase como en la figura 2.3, donde aparecen dos animales diferentes, la clasificación multi-clase provocaría una pérdida de información. Debido a esto la clasificación multi-etiqueta es un campo que se ha desarrollado mucho en los últimos años.



Figura 2.3: Clasificación multi-etiqueta

2.2.1. Descripción formal del MLC

Siendo $\mathcal{D}$ un dataset multi-etiquetas compuesto por un conjunto de $\Updownarrow$ instancias, y definido como $\mathcal{D} = \{(x_i, Y_i)|1 \leq i \leq m\}$. Siendo $\mathcal{X} = X_1 \times \cdots \times X_d$ el espacio de entrada de dimensión d, e $\mathcal{Y} = \{\lambda_1, \lambda_2, \ldots, \lambda_q\}$ el espacio compuesto por $q > 1$ etiquetas. Cada instancia multi-etiqueta está compuesta por un vector de entrada xi y un conjunto de etiquetas relevantes asociadas a él $Y_i \subseteq Y$. Tener en cuenta que cada Y diferente también recibe el nombre de conjunto de etiquetas. En la figura 2.4, podemos ver un ejemplo de un conjunto de datos multi-etiqueta. Como podemos observar cada ejemplo está etiquetado con una o más etiquetas de manera simultánea.

Instance	Features	Labels				
		λ_1	λ_2	λ_3	...	λ_q
#1	$\mathbf{x}_1$	0	1	0	...	0
#2	$\mathbf{x}_2$	1	0	1	...	1
#3	$\mathbf{x}_3$	0	1	0	...	1
#4	$\mathbf{x}_4$	0	1	0	...	0
#5	$\mathbf{x}_5$	0	1	1	...	0
...	...			...		
#m	$\mathbf{x}_m$	1	0	1	...	1

Figura 2.4: Ejemplo de conjunto de datos multi-etiqueta obtenido de [1]

El objetivo del MLC es construir un modelo predictivo $h : \mathcal{X} \to 2^{\mathcal{Y}}$ que ofrezca un conjunto de etiquetas relevantes para una instancia desconocida. Por tanto, para cada $x \in \mathcal{X}$, una bipartición $\hat{b} = (\hat{Y}, \overline{\hat{Y}})$ del espacio de etiquetas $\mathcal{Y}$ es dado, donde $\hat{Y} = h(x)$ es el conjunto de etiquetas relevantes e $\overline{\hat{Y}}$ el conjunto de irrelevantes. Esta bipartición también se puede dar como un vector binario $\hat{b} = \{0, 1\}^q$, indicando si cada etiqueta es relevante (1) o no (0).

2.2.2. Eligiendo modelo para clasificación multi-etiqueta

Para la implementación de un clasificador multi-etiqueta debemos elegir qué arquitectura de red usaremos. Vamos a revisar las opciones que tenemos, por ejemplo en [9] se analiza el cambio de paradigma en el problema que han traído los sistemas basados en redes convolucionales y transformers y estudia como diferentes métodos derivados de estas redes han mejorado la calidad de los sistemas de aprendizaje para esta tarea. Podemos encontrar un benchmarking de diferentes arquitecturas de redes como ResNet101 [10], ViT-Base [11], ML-GCN [12], ASL [13], Q2L [14], SALGL [15], SGRE [16], and PAT [17]. Por simplicidad y su uso tan extendido usaremos ResNet en nuestra implementación.

En cuanto a la resolución del problema conjunto de crear un clasificador multi-etiqueta dentro de un entorno federado, como ya hemos visto anterioremente hay estudios que se centran en evitar el problema haciendo un tratamiento previo de los datos para evitar el desbalanceo, otros estudian diferentes arquitecturas de la red como en [18]. Sin embargo estos estudios comparten la misma red entre los clientes o se aplican sobre datos tabulados. Nuestra propuesta requiere que la distribución de los datos contenga diferente número de etiquetas y que el sistema acepte una arquitectura con diferentes redes. Tras una búsqueda bibliográfica, no hemos encontrado ningún estudio que una todos estos conceptos en una solución.

3. Federación con capas personalizadas y salidas heterogéneas (FedPerH)

En esta sección vamos a proponer una solución al problema de clasificación multi-etiqueta en entornos federados. Veremos cómo trataremos los datos del dataset para adaptarlos a nuestro entorno federado, seleccionaremos la herramienta para el sistema federado, el modelo a usar y las modificaciones pertinentes. Por último definiremos las métricas elegidas para evaluar la bondad del sistema.

3.1. Datos de entrenamiento

Nuestro objetivo es tener un número de distintos conjuntos de datos con sus propias etiquetas no compartidas con los demás, buscamos simular distintas tareas a entrenar que sean específicas para cada cliente. Para ello no solo vamos a separar los diferentes ejemplos por cada cliente sino que también tenemos que hacer una discriminación por etiquetas.

Como base vamos a usar el conjunto de imágenes multi-etiqueta Corel-5k, que tiene 5000 imágenes de 50 categorías y un diccionario de 260 palabras o clases [1]. A partir de él crearemos 4 conjuntos de datos con 10 etiquetas diferentes cada uno ya que vamos a usar 4 clientes en nuestra federación, para hacer la separación por etiquetas vamos a ordenar las 40 que más se repiten, creamos cuatro grupos de 10 etiquetas de manera que el número de ejemplos de cada grupo sea similar. Una vez tenemos los 4 grupos seleccionamos del dataset total los ejemplos correspondientes a las etiquetas de cada grupo, por ejemplo si en nuestro grupo tenemos "agua" cogemos todos los elementos del conjunto de datos que estén etiquetados con "agua", luego eliminamos de esos ejemplos todas las etiquetas que no corres-

[1] `https://www.kaggle.com/datasets/parhamsalar/corel5k`

pondan a su grupo. Hacemos esto para los 4 subsets y así conseguimos separar las imágenes de manera que cada conjunto solo contenga etiquetas propias y que el número de ejemplos sea parecido.

Una vez hemos conseguido fabricar unos conjuntos de datos que nos permitan simular una situación real en la que tenemos conjuntos de datos distintos que quieren entrenar un mismo modelo lo siguiente es preparar el sistema que va a procesar esos datos.

3.2. Arquitectura de la solución

Vamos a montar un sistema que nos permita comparar el rendimiento de un clasificador multi-etiqueta en un entorno federado y uno centralizado usando los mismos parámetros para ver si existe mejora en la tarea al pertenecer a un grupo colaborativo de entrenamiento. Tendremos 4 clientes y cada uno realizará una tarea de clasificación distinta ya que cada conjunto de datos va a tener etiquetas distintas. Para que realicen esta tarea vamos a usar modelos personalizados que contarán con una parte común que es la que se compartirá con el servidor y un conjunto de capas personalizadas que quedarán en los clientes. De esta manera parte del modelo tendrá una función general y compartida entre los demás nodos y cada uno de ellos entrenará la parte específica a su tarea de manera local. Los clientes compartirán la parte común del modelo y el servidor se encargará de realizar la agregación y de devolverle esa parte actualizada todos ellos.

3.2.1. FedPer

El aprendizaje federado necesita una insfraestructura que permita mover modelos de un lado a otro entre el servidor y los clientes, ejecutar el entrenamiento con los datos locales y agregar los modelos actualizados. Para evitar la implementación de todo el sistema desde cero vamos a usar de base el entorno de trabajo para aprendizaje federado Flower [2] que nos provee de esta infraestructura. Más en concreto haremos uso del baseline de Flower FedPer [3], una implementación de [8] el cual propone una aplicación del método de *Backbone Sharing* que vimos en [4]. FedPer es un entorno simulado que crea el servidor y los clientes, en el servidor se instancia el modelo global con dos partes diferenciadas llamadas *cuerpo* y *cabeceras*, el *cuerpo* corresponde a la parte del modelo que va a ser compartida y las *cabeceras* son las capas que van a ser entrenadas en el cliente pero no van a devolverse. Se inicializa el *cuerpo* y se envía a cada uno de los clientes los cuales los restauran en sus modelos locales. Cada modelo local se entrena con los sus datos

[2] `https://flower.ai/`

[3] `https://flower.ai/docs/baselines/fedper.html`

y los *cuerpos* actualizados se devuelven al servidor el cuál realiza la operación de agregación que en nuestro caso será FedAvg [3] pero solo con la parte del modelo enviada por los clientes. Una vez actualizado el modelo global se devuelve a los clientes para la siguiente ronda de entrenamiento.

FedPer y Flower también nos dan herramientas para la agregación de métricas de entrenamiento y evaluación.

3.2.2. Modelo

Para las capas compartidas con el servidor necesitamos un modelo que nos ayude con la tarea de procesamiento de imágenes. Para nuestra implementación vamos a usar la red pre-entrenada ResNet [10], por simplificar el entrenamiento y no partir desde cero. También es conocida su eficiencia para la tarea de clasificación de imágenes, reconocimiento de objetos y extracción de características haciéndola una muy buena candidata para la tarea común de procesamiento de imágenes gracias a su capacidad de realizar conexiones residuales entre sus capas haciendo que se pueda incrementar el tamaño de la red sin que afecte al gradiente. Esta parte de la red se encargará de de las funciones comunes a las cuatro tareas y las capas personalizadas de cada cliente se encargarán de la parte específica. Para las capas personalizadas vamos a añadir para cada servidor dos capas convolucionales y una capa linear para la salida con 10 nodos, uno para cada clase.

3.2.3. Modificaciones

Como comentamos antes, la implementación de FedPer que estamos usando está implementada para clasificación multi-etiqueta. Debemos adaptar la arquitectura para poder implementar la clasificación multi-etiqueta. Para ello hemos realizado modificaciones en la parte de la preparación de los datos, los clientes, el modelo y el tratamiento de métricas.

Para el tratamiento de datos hemos tenido que crear el conjunto de datos desde cero como explicamos en la sección 3.1, definimos una función que creará los 8 Dataloaders que usará cada cliente, 4 para entrenamiento y 4 para evaluación. Cada separación de datos tiene alrededor de 4400 ejemplos y unas 5000 etiquetas. Hemos codificado las etiquetas de los dataset en one-hot para facilitar el entrenamiento y la evaluación multi-etiqueta.

Los clientes al inicializarse reciben los 8 Dataloaders y en función del identificador del cliente eligen uno de los conjuntos de datos que luego enviarán al modelo para que realice en entrenamiento y la evaluación.

El modelo inicializa las capas personalizadas con valores distintos para cada cliente y carga los datos específicos de ese cliente. En el entrenamiento usamos como función de pérdida BCEWithLogits en lugar de CrossEntrophyLoss ya que está tiene buenos resultados a la hora del entrenamiento de clasificadores multi-etiqueta. BCEWithLogits permite introducir los pesos de cada etiqueta en un tensor para incorporar el desbalance de etiquetas al cálculo de la función de pérdida. Para la precisión del entrenamiento hemos usado [4].

3.3. Métricas de evaluación

A la hora de comprobar la bondad de nuestro modelo tenemos un problema añadido cuando la salida son más de una etiqueta. Por ejemplo, si el resultado esperado tiene dos clases activas y el modelo solo ha predicho una de ellas, si comprobamos el valor exacto de todas las etiquetas marcaríamos la precisión como 0, sin embargo hay una que sí ha sido correcta y la debemos evaluar de forma positiva. Podríamos comprobar etiqueta a etiqueta si tiene un valor correcto pero si la mayoría de valores en las etiquetas son 0, un modelo que prediga siempre 0 podría tener una precisión muy alta y no funcionaría correctamente. Por eso debemos buscar métricas que tengan en cuenta los aciertos y los fallos de manera global.

3.3.1. Label ranking average precision

Necesitamos una métrica de precisión para el entrenamiento, para evitar la complicación de hacer los cálculos de la matriz de confusión vamos a usar la métrica implementada en scikit learn *Label ranking average precision score* ordena las predicciones según el valor de probabilidad asignado por el modelo y responde a la pregunta para cada etiqueta verdadera, ¿qué fracción de las etiquetas predichas con valro más alto son correctas?

The label ranking average precision score function implements label ranking average precision (LRAP). This metric is linked to the average precision score function, but is based on the notion of label ranking instead of precision and recall. Esta métrica será mayor si el modelo es capaz de dar una mayor probabilidad a las etiquetas verdaderas.

[4] `https://scikit-learn.org/stable/modules/model_evaluation.html#label-ranking-average-precision`

3.3.2. Matriz de confusión

Una matriz de confusión distribuye las predicciones en base a la salida esperada de la siguiente manera:

1. Verdaderos positivos (TP): Clases predichas correctamente
2. Verdaderos negativos (TN): Ausencia de clase predicha correctamente
3. Falsos positivos (FP): Clase predicha incorrectamente
4. Falsos negativos (FN): Ausencia de clase predicha incorrectamente

Distribuidas de la siguiente manera:

$$\begin{bmatrix} TN & FP \\ TP & FN \end{bmatrix}$$

Imaginemos que tenemos tres imágenes con las siguientes etiquetas:

water	buildings	rocks	field	sand	cars	polar	ruins	statue	wall
1	0	1	0	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0
1	0	1	0	0	0	0	0	0	0

Y el modelo realiza las siguiente predicciones:

water	buildings	rocks	field	sand	cars	polar	ruins	statue	wall
1	0	0	0	0	1	0	0	0	0
0	1	0	0	0	1	0	0	0	0
0	0	0	0	0	1	0	0	0	0

Las matrices de confusión para cada etiqueta serían

Water:

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

Buildings:

$$\begin{bmatrix} 2 & 0 \\ 1 & 0 \end{bmatrix}$$

Rocks:

$$\begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$$

Cars:

$$\begin{bmatrix} 0 & 2 \\ 1 & 0 \end{bmatrix}$$

Antes de crear nuestra matriz de confusión debemos binarizar nuestras salidas del modelo. Una vez tenemos nuestra matriz de confusión para cada ejemplo del *batch* de evaluación podremos calcular el resto de las métricas.

Accuracy

Accuracy is the proportion of examples that were correctly classified. More precisely, it is sum of the number of true positives and true negatives, divided by the number of examples in the dataset.

For example, if we look at the cat class, we'll see that among 4 training examples in the dataset, the prediction of the model for the class cat was correct in 2 of them. So accuracy for class cat is 2 / 4 = 0.5 = 50

$$Accuracy = \frac{TP + TN}{FP + FN + TP + TN}$$

Precision

Precision is the proportion of correct predictions among all predictions of a certain class. In other words, it is the proportion of true positives among all positive predictions.

For example, if we look at the cat class, the number of times the model predicted a cat is 2, and only one of them was a correct prediction. Therefore the precision would be 1 / 2 = 0.5 = 50

$$Precision = \frac{TP}{FP + TP}$$

Recall

Recall is the proportion of examples of a certain class that have been predicted by the model as belonging to that class. In other words, it is the proportion of true positives among all true examples.

For example, if we look at the dog class, we'll see that the number of dog examples in the dataset is 1, and the model did classify that one correctly. Therefore the recall for the dog class would be 1 / 1 = 1 = 100

La sensibilidad nos indica la capacidad de nuestro estimador para dar como casos positivos (Fig.→ círculo grande) los casos realmente enfermos (Fig.→ punto); proporción de enfermos correctamente identificados. Es decir, la sensibilidad caracteriza la capacidad de la prueba para detectar la enfermedad en sujetos enfermos.

$$Recall = \frac{TP}{FN + TP}$$

Specificity

Tasa de verdaderos negativos

La especificidad nos indica la capacidad de nuestro estimador para dar como casos negativos (Fig.→ rectángulo grande) los casos realmente sanos (Fig.→ aro); proporción de sanos correctamente identificados. Es decir, la especificidad caracteriza la capacidad de la prueba para detectar la ausencia de la enfermedad en sujetos sanos.

$$Specificity = \frac{TN}{TN + FP}$$

Fall out

Tasa de falsos positivos

probability of falsely rejecting the null hypothesis for a particular test. The false positive rate is calculated as the ratio between the number of negative events wrongly categorized as positive (false positives) and the total number of actual negative events (regardless of classification).

The false positive rate (FPR) is the proportion of all negatives that still yield positive test outcomes, i.e., the conditional probability of a positive test result given an event that was not present.

$$Fallout = \frac{FP}{FP + TN}$$

Miss rate

Tasa de falsos negativos

Complementarily, the false negative rate (FNR) is the proportion of positives which yield negative test outcomes with the test, i.e., the conditional probability of a negative test result given that the condition being looked for is present.

$$Fallout = \frac{FN}{FN + TP}$$

3.3.3. F1 Score

The F1 score for a certain class is the harmonic mean of its precision and recall, so it's an overall measure of the quality of a classifier's predictions. It is usually the metric of choice for most people because it captures both precision and recall.

$$F_1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

COMMENTARIOS:

Jose, podría añadir descripciones formales de:

FedPer (https://arxiv.org/pdf/1912.00818) BCEWithLogits (https://docs.pytorch.org/docs/2.14/ger
Label ranking average precision (https://scikit-learn.org/stable/modules/model$_evaluation.html label-ranking-average-precision$)

Pero no sé si aportan algo en realidad.

4. Estudio experimental

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Mauris id tortor eget tellus sollicitudin molestie. Morbi vehicula aliquam turpis, eget efficitur dolor rhoncus non. Integer mi odio, elementum non euismod sed, elementum eu dolor. Maecenas commodo sapien sed ultricies posuere. Ut molestie, mauris et tempor fermentum, ex massa consequat elit, eu consequat justo nibh ut augue. Sed in justo dui. Sed non molestie nisi. Ut fermentum nec ante sed ultrices. Vestibulum placerat quam urna, nec sollicitudin nibh rhoncus non. Curabitur pulvinar facilisis posuere.

4.1. Configuración de los experimentos

Donec eu dolor et quam porta vulputate vitae vulputate augue. Etiam fermentum, lectus vitae euismod feugiat, elit ligula luctus ante, eget commodo turpis leo vel quam. Aenean consectetur in turpis sit amet varius. Sed porta porttitor lectus, id tempus nunc suscipit vitae. Etiam blandit quam quis magna ornare pharetra. Aenean eget tortor nisi. Sed sit amet tempus eros. Phasellus pretium, nunc nec fringilla placerat, metus lectus maximus eros, nec condimentum neque ex nec orci. Vivamus facilisis at turpis sit amet maximus. Ut dolor justo, bibendum eget rhoncus eget, malesuada malesuada lacus. Suspendisse potenti.

Donec augue nulla, semper ac tempor sit amet, vehicula sed est. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Pellentesque vel convallis justo, quis feugiat velit. Etiam fermentum, risus et venenatis rhoncus, ipsum ligula congue neque, sed consectetur arcu turpis vel metus. Integer ornare turpis et luctus ullamcorper. Ut placerat a ex et consequat. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas.

4.2. Análisis de resultados

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Mauris id tortor eget tellus sollicitudin molestie. Morbi vehicula aliquam turpis, eget efficitur dolor rhoncus non. Integer mi odio, elementum non euismod sed, elementum eu dolor. Maecenas commodo sapien sed ultricies posuere. Ut molestie, mauris et tempor fermentum, ex massa consequat elit, eu consequat justo nibh ut augue. Sed in justo dui. Sed non molestie nisi. Ut fermentum nec ante sed ultrices. Vestibulum placerat quam urna, nec sollicitudin nibh rhoncus non. Curabitur pulvinar facilisis posuere.

Donec eu dolor et quam porta vulputate vitae vulputate augue. Etiam fermentum, lectus vitae euismod feugiat, elit ligula luctus ante, eget commodo turpis leo vel quam. Aenean consectetur in turpis sit amet varius. Sed porta porttitor lectus, id tempus nunc suscipit vitae. Etiam blandit quam quis magna ornare pharetra. Aenean eget tortor nisi. Sed sit amet tempus eros. Phasellus pretium, nunc nec fringilla placerat, metus lectus maximus eros, nec condimentum neque ex nec orci. Vivamus facilisis at turpis sit amet maximus. Ut dolor justo, bibendum eget rhoncus eget, malesuada malesuada lacus. Suspendisse potenti.

Donec augue nulla, semper ac tempor sit amet, vehicula sed est. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Pellentesque vel convallis justo, quis feugiat velit. Etiam fermentum, risus et venenatis rhoncus, ipsum ligula congue neque, sed consectetur arcu turpis vel metus. Integer ornare turpis et luctus ullamcorper. Ut placerat a ex et consequat. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas.

5. Conclusiones y trabajo futuro

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Mauris id tortor eget tellus sollicitudin molestie. Morbi vehicula aliquam turpis, eget efficitur dolor rhoncus non. Integer mi odio, elementum non euismod sed, elementum eu dolor. Maecenas commodo sapien sed ultricies posuere. Ut molestie, mauris et tempor fermentum, ex massa consequat elit, eu consequat justo nibh ut augue. Sed in justo dui. Sed non molestie nisi. Ut fermentum nec ante sed ultrices. Vestibulum placerat quam urna, nec sollicitudin nibh rhoncus non. Curabitur pulvinar facilisis posuere.

5.1. Conclusiones

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Mauris id tortor eget tellus sollicitudin molestie. Morbi vehicula aliquam turpis, eget efficitur dolor rhoncus non. Integer mi odio, elementum non euismod sed, elementum eu dolor. Maecenas commodo sapien sed ultricies posuere. Ut molestie, mauris et tempor fermentum, ex massa consequat elit, eu consequat justo nibh ut augue. Sed in justo dui. Sed non molestie nisi. Ut fermentum nec ante sed ultrices. Vestibulum placerat quam urna, nec sollicitudin nibh rhoncus non. Curabitur pulvinar facilisis posuere.

Donec eu dolor et quam porta vulputate vitae vulputate augue. Etiam fermentum, lectus vitae euismod feugiat, elit ligula luctus ante, eget commodo turpis leo vel quam. Aenean consectetur in turpis sit amet varius. Sed porta porttitor lectus, id tempus nunc suscipit vitae. Etiam blandit quam quis magna ornare pharetra. Aenean eget tortor nisi. Sed sit amet tempus eros. Phasellus pretium, nunc nec fringilla placerat, metus lectus maximus eros, nec condimentum neque ex nec orci. Vivamus facilisis at turpis sit amet maximus. Ut dolor justo, bibendum eget rhoncus eget, malesuada malesuada lacus. Suspendisse potenti.

Donec augue nulla, semper ac tempor sit amet, vehicula sed est. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Pellentesque vel convallis justo, quis feugiat velit. Etiam fermentum, risus et venenatis rhoncus, ipsum ligula congue neque,

sed consectetur arcu turpis vel metus. Integer ornare turpis et luctus ullamcorper. Ut placerat a ex et consequat. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas.

5.2. Trabajo futuro

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Mauris id tortor eget tellus sollicitudin molestie. Morbi vehicula aliquam turpis, eget efficitur dolor rhoncus non. Integer mi odio, elementum non euismod sed, elementum eu dolor. Maecenas commodo sapien sed ultricies posuere. Ut molestie, mauris et tempor fermentum, ex massa consequat elit, eu consequat justo nibh ut augue. Sed in justo dui. Sed non molestie nisi. Ut fermentum nec ante sed ultrices. Vestibulum placerat quam urna, nec sollicitudin nibh rhoncus non. Curabitur pulvinar facilisis posuere.

Donec eu dolor et quam porta vulputate vitae vulputate augue. Etiam fermentum, lectus vitae euismod feugiat, elit ligula luctus ante, eget commodo turpis leo vel quam. Aenean consectetur in turpis sit amet varius. Sed porta porttitor lectus, id tempus nunc suscipit vitae. Etiam blandit quam quis magna ornare pharetra. Aenean eget tortor nisi. Sed sit amet tempus eros. Phasellus pretium, nunc nec fringilla placerat, metus lectus maximus eros, nec condimentum neque ex nec orci. Vivamus facilisis at turpis sit amet maximus. Ut dolor justo, bibendum eget rhoncus eget, malesuada malesuada lacus. Suspendisse potenti.

Bibliografía

[1] Jose M. Moyano. Multi-label classification models for heterogeneous data: an ensemble-based approach, 2020.

[2] Qiang Yang, Yang Liu, Yong Cheng, Yan Kang, Tianjian Chen, and Han Yu. *Federated Learning*. Morgan & Claypool publishers, 2019.

[3] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. 2023.

[4] Mang Ye, Xiuwen Fang, Bo Du, Pong C. Yuen, and Dacheng Tao. Heterogeneous federated learning: State-of-the-art and research challenges. *ACM Comput. Surv.*, 56(3), October 2023.

[5] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, Damon Civin, and Vikas Chandra. Federated learning with non-iid data. 2018.

[6] Marcos F. Criado, Fernando E. Casado, Roberto Iglesias, Carlos V. Regueiro, and Senén Barro. Non-iid data and continual learning processes in federated learning: A long road ahead. 2021.

[7] Zhaobin Sun, Nannan Wu, Junjie Shi, Li Yu, Xin Yang, Kwang-Ting Cheng, and Zengqiang Yan. Fedmlp: Federated multi-label medical image classification under task heterogeneity. 2024.

[8] Manoj Ghuhan Arivazhagan, Vinay Aggarwal, Aaditya Kumar Singh, and Sunav Choudhary. Federated learning with personalization layers. 2019.

[9] Xuelin Zhu, Xiu-Shen Wei, Jiawei Ge, Shuai Xu, and Bing Wang. Rethinking multi-label image classification with deep learning: Taxonomy, challenge, and outlook. 2026.

[10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. 2015.

[11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,

Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. 2021.

[12] Zhao-Min Chen, Xiu-Shen Wei, Peng Wang, and Yanwen Guo. Multi-label image recognition with graph convolutional networks. 2019.

[13] Emanuel Ben-Baruch, Tal Ridnik, Nadav Zamir, Asaf Noy, Itamar Friedman, Matan Protter, and Lihi Zelnik-Manor. Asymmetric loss for multi-label classification. 2021.

[14] Shilong Liu, Lei Zhang, Xiao Yang, Hang Su, and Jun Zhu. Query2label: A simple transformer way to multi-label classification. 2021.

[15] Xuelin Zhu, Jian Liu, Weijia Liu, Jiawei Ge, Bo Liu, and Jiuxin Cao. Scene-aware label graph learning for multi-label image classification. pages 1473–1482, 2023.

[16] Xuelin Zhu, Jianshu Li, Jiuxin Cao, Dongqi Tang, Jian Liu, and Bo Liu. Semantic-guided representation enhancement for multi-label image classification. *IEEE Transactions on Circuits and Systems for Video Technology*, 34(10):10036–10049, 2024.

[17] Ming-Kun Xie, Jia-Hao Xiao, Pei Peng, Gang Niu, Masashi Sugiyama, and Sheng-Jun Huang. Counterfactual reasoning for multi-label image classification via patching-based training. 2024.

[18] Barış Büyüktaş, Kenneth Weitzel, Sebastian Völkers, Felix Zailskas, and Begüm Demir. Transformer-based federated learning for multi-label remote sensing image classification. 2024.